Edu SaaS Backend API Documentation

Generated from the repository code on 2026-06-10. This documentation covers only endpoints mounted in `src/app.js`, `src/routes/authRoutes.js`, and `src/routes/institutionRoutes.js`. When exact behavior is broad or product-dependent, it is marked TODO/VERIFY instead of guessed.

Overview

This is a Node.js + Express + Mongoose multi-tenant Edu SaaS backend for school and university institutions.

Mounted API surface:

- `POST /stripe/webhook` is mounted before JSON parsing and uses `express.raw({ type: "application/json" })`.
- `/auth` is mounted from `src/routes/authRoutes.js`.
- `/institutions` is mounted from `src/routes/institutionRoutes.js`.
- `/uploads` is static public storage for institution logos and member profile pictures only.

Route files inspected:

- `src/app.js`
- `src/server.js`
- `src/routes/authRoutes.js`
- `src/routes/institutionRoutes.js`

Supporting code inspected:

- Controllers, services, middleware, permission catalogs, scoped responsibility helpers, models, upload middleware, tests, Postman files, package/env references, and error handlers.

Authentication Flow

JWT authentication uses `Authorization: Bearer <JWT_TOKEN>`.

`authJwt`:

- Reads the bearer token.
- Verifies it with `JWT_SECRET`.
- Requires `payload.sub` to be a valid user id.
- Loads the user and rejects missing or inactive accounts.

- Sets `req.user = { id, email, status, isEmailVerified }`.

`requireEmailVerified`:

- Requires `req.user`.
- Rejects unverified accounts with `403 { "message": "Email is not verified" }`.

Auth flow:

1. `POST /auth/signup` creates a user, institution, admin membership, sends signup OTP, and returns a JWT.
2. Signup-created users are not fully usable for tenant routes until OTP verification.
3. `POST /auth/otp/verify` verifies email for the authenticated user.
4. `POST /auth/login` requires an active verified email and returns a JWT.
5. Forgot/reset password uses OTP and generic forgot-password messaging.

Tenant / Institution Scoping Rules

Tenant-owned business routes live under `/institutions/:institutionId/...`.

`institutionAccess()`:

- Reads `req.params.institutionId`, falling back to `x-institution-id` only when no path param exists.
- Requires the authenticated user to have an active membership in that institution.
- Sets `req.tenantId` and `req.membership`.

Scoping rules:

- Institution-owned data must be queried with `institutionId` plus resource ids.
- Do not fetch/update/delete tenant-owned records by `_id` alone.
- `GET /institutions/me` and `POST /institutions` are authenticated account-level routes and do not use a tenant path param.
- Public/global endpoints are auth signup/login/password flows and the Stripe webhook.

Role And Permission Model

`ROLE_PERMISSIONS` from `src/auth/permissions.js`:

Role	Permissions
admin	institution.manage, department.manage, class.manage, membership.manage, semester.manage, subject.manage, section.read, section.manage, student.kick, attendance.manage, billing.manage
instructor	section.read, attendance.take, attendance.excuse

Role	Permissions
student	section.read, enrollment.self_manage, attendance.read_self, billing.read_self

Notes:

- `Membership.role` values are `admin`, `instructor`, and `student`.
- Institution ownership is checked in selected services in addition to membership role.
- Some section-linked routes rely on service-level authorization instead of route-level `requirePermission(...)`.
- Students have `section.read` for student-facing section-linked flows, but the generic staff section list/detail service blocks student memberships.

Responsibility / Scoped Permission Model

Responsibilities are scoped assignments, not `Membership.role` values. The backend catalog is the source of truth for effective responsibility permissions; raw stored `ResponsibilityAssignment.permissions` are not trusted for authorization.

`RESPONSIBILITY_PERMISSIONS`:

Responsibility	Effective permissions
head	academicRisk.read_scope, academicProfile.read_scope, academicStanding.read_scope
coordinator	section.read_scope, sectionSession.read_scope, sectionSession.manage_scope, attendance.read_scope, attendance.manage_scope
mentor	none currently active

Allowed responsibility scopes:

Responsibility	Allowed `scopeType` values
head	institution, department, classLevel
coordinator	department, classLevel, class, section
mentor	class, section

Assignments must be active, within optional `startAt` / `endAt`, same tenant, and valid for their responsibility/scope combination.

Head assignments provide read-only academic oversight only. Head does not receive section/session/attendance/enrollment/assessment/result/billing/membership write or operational permissions.

Coordinator assignments are currently used by selected section reads, section-session reads/management, and attendance reads/management.

Mentor assignments have no effective operational or academic permissions yet.

Do not activate future scoped permissions such as `studentProfile.read_scope`, `attendance.excuse_manage_scope`, `academicDecision.review_scope`, `enrollment.review_scope`, or `studentDropRisk.review_scope` until service-level enforcement exists.

Common Headers

Header	Required for	Notes
`Authorization: Bearer <JWT_TOKEN>`	Protected routes	Required by `authJwt`.
`Content-Type: application/json`	JSON request bodies	Default for create/update APIs.
`Content-Type: multipart/form-data`	Upload routes	See file notes below.
`Stripe-Signature`	`POST /stripe/webhook`	Required for Stripe signature verification.
`x-institution-id`	Rare fallback	Supported by `institutionAccess()`, but current tenant routes use `:institutionId`.

Common Response Envelope Shape

Controllers return service envelopes directly. Common shapes are:

```
{ "resourceName": { "id": "507f1f77bcf86cd799439011" } }
```

```
{ "resources": [], "pagination": { "page": 1, "limit": 20, "total": 0, "totalPages": 0 } }
```

```
{ "message": "Operation completed successfully" }
```

Common envelope keys include `user`, `institution`, `membership`, `department`, `classLevel`, `schoolClass`, `assignment`, `subject`, `semester`, `academicYear`, `member`, `section`, `sectionSession`, `attendanceRecord`, `attendanceRecords`, `attendanceRoster`, `attendanceAnalytics`, `assessment`, `assessmentResults`, `homework`, `submission`, `material`, `academicPolicy`, `schoolAcademicPolicy`, `academicProfile`, `academicRiskSummary`, `reportCard`, `schoolFinalOutcome`, `promotionSummary`, `sectionOutcomeSummary`, `standingSummary`, `responsibilityAssignment`, `responsibilityAssignments`, `students`, `billingProfile`, and `checkoutSession`.

Common Error Response Shape

All route/controller error responses use:

```
{ "message": "..." }
```

Common statuses:

Status	Meaning
400	Invalid ObjectId/body/query/upload, missing required field, invalid lifecycle field.
401	Missing/invalid/expired JWT, invalid credentials, unauthorized context.
403	Inactive account, unverified email, no tenant access, permission denied.
404	Tenant-scoped resource or active context not found.
409	Duplicate/conflict, invalid state transition, full capacity, already open/closed, wrong institution type.
429	OTP max attempts exceeded.
500	Internal errors; message is masked as `Internal server error`.
502	Stripe checkout/session failure.

File Upload / Download Notes

Use case	Field name	Storage	Accepted types	Size limit	Public
Institution logo	`logo`	`uploads/institution-logos`	image/jpeg, image/png, image/webp	Default 2MB, `MAX_LOGO_SIZE_MB` override	Yes
Member profile picture	`profilePicture`	`uploads/member-profiles`	image/jpeg, image/png, image/webp	Default 2MB, `MAX_PROFILE_PICTURE_SIZE_MB` override	Yes
Homework submission	`submissionFile`	`storage/homework-submissions`	application/pdf plus `.pdf` extension	Default 10MB, `MAX_HOMEWORK_SUBMISSION_SIZE_MB` override	No
Section material	`materialFile`	`storage/section-materials`	application/pdf plus `.pdf` extension	Default 10MB, `MAX_SECTION_MATERIAL_SIZE_MB` override	No

Private files are served only through protected download routes after service-level authorization. `storage/` is not exposed by `express.static`.

TODO/VERIFY: Current PDF validation is MIME/extension and service-level metadata/path safety; PDF magic-byte validation is deferred.

Environment / Base URL Notes

Default local base URL is `http://localhost:5000` unless `PORT` is set.

Environment variable names found in code/docs:

- `MONGO_URI`, `PORT`, `NODE_ENV`
- `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`
- `OTP_EXPIRES_MINUTES`, `OTP_MAX_ATTEMPTS`
- `GMAIL_USER`, `GMAIL_APP_PASSWORD`, `EMAIL_FROM`
- `STRIPE_SECRET_KEY`, `STRIPE_PRICE_BASIC_MONTHLY`, `STRIPE_WEBHOOK_SECRET`, `BILLING_SUCCESS_URL`, `BILLING_CANCEL_URL`
- `MAX_LOGO_SIZE_MB`, `MAX_PROFILE_PICTURE_SIZE_MB`, `MAX_HOMEWORK_SUBMISSION_SIZE_MB`, `MAX_SECTION_MATERIAL_SIZE_MB`
- `ATTENDANCE_QR_DATE_OVERRIDE` for non-production local attendance QR testing only

Do not document or expose real secret values.

Full Endpoint Reference

See API_ENDPOINT_INDEX.md for the compact all-endpoint table. The grouped reference below adds request, validation, response, security, and TODO/VERIFY notes by module. All paths are full API paths.

Auth

Method	Path	Purpose	Auth	Request / validation	Success	Errors / notes
POST	`/auth/signup`	Create user, institution, admin membership, send signup OTP.	Public	JSON `email`, `password`, `confirmPassword`, `institutionName`, `institutionType` (`school` or `university`). Password >= 8, email unique.	`201 { accessToken, user, institution, membership }`	`409` duplicate email; sends OTP; email must be verified before tenant routes.
POST	`/auth/login`	Login active verified user.	Public	JSON `email`, `password`.	`200 { accessToken, user }`	`401` invalid credentials; `403` inactive/unverified.
POST	`/auth/password/forgot`	Send password reset OTP.	Public	JSON `email`; email format validated.	`200 { message }`	Generic response for unknown/ineligible email.
POST	`/auth/password/reset`	Reset password using OTP.	Public	JSON `email`, `code`, `newPassword`.	`200 { message }`	OTP expiry/max attempts enforced; new password must differ from current.
POST	`/auth/otp/verify`	Verify signup OTP.	`authJwt`	JSON `code`.	`200 { message, user }`	Does not use `requireEmailVerified`; returns already-verified message if applicable.
POST	`/auth/otp/resend`	Resend signup OTP.	`authJwt`	No body.	`200 { message }`	`409` if already verified.

Institutions

Method	Path	Purpose	Auth / middleware	Request / validation	Success	Errors / notes
GET	`/institutions/me`	Current user institution context.	`authJwt`, `requireEmailVerified`	No params/body.	`200 { user, memberships, activeInstitution, nextStep }`	Global account context; not tenant-scoped by path.
POST	`/institutions`	Create additional institution.	`authJwt`, `requireEmailVerified`	JSON `name`, `type` (`school` or `university`).	`201 { institution, membership }`	Creates active admin membership for same user.
PATCH	`/institutions/:institutionId`	Update institution.	`authJwt`, `requireEmailVerified`, `institutionAccess()`, `requirePermission("institution.manage")`	Path `institutionId`; JSON patch accepted by `institutionService`.	`200 { institution }`	Tenant-scoped. TODO/VERIFY exact patchable policy fields before generated clients.
POST	`/institutions/:institutionId/logo`	Upload institution logo.	Same plus `uploadInstitutionLogo`	Path `institutionId`; multipart field `logo`.	`200 { institution }`	Public file URL under `/uploads/institution-logos`; JPG/PNG/WEBP only.

Billing / Stripe

Method	Path	Purpose	Auth / permission	Request	Success	Security notes
POST	`/institutions/:institutionId/billing/checkout-session`	Create Stripe-hosted subscription Checkout Session.	`institutionAccess()`; service requires owner or active admin.	No body.	`201 { checkoutSession: { id, url } }`	Uses `STRIPE_PRICE_BASIC_MONTHLY`; rejects duplicate active/trialing/past_due subscriptions with subscription id.
GET	`/institutions/:institutionId/billing/status`	Read local billing profile.	`institutionAccess()`	No body.	`200 { billingProfile }`	Returns local Stripe ids/status projection only; no card/payment method data.
POST	`/stripe/webhook`	Process Stripe webhook.	Public route, Stripe signature.	Raw `application/json` body with `Stripe-Signature`.	`200 { received, duplicate }`	Must remain before `express.json()`; verifies with `STRIPE_WEBHOOK_SECRET`; idempotent via `StripeWebhookEvent`.

Responsibility Assignments

Owner/admin management for scoped Head, Coordinator, and Mentor assignments. These routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`; the service performs the owner/admin check. A scoped Head/Coordinator/Mentor assignment alone cannot manage responsibility assignments.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/responsibility-assignments`	JSON `userId`, `responsibilityType`, `scopeType`, `scopeId`, optional `startAt`, `endAt`, `isActive`.	`201 { responsibilityAssignment }`	Owner/admin only. Target user must have active same-institution instructor membership.
GET	`/institutions/:institutionId/responsibility-assignments`	Query `userId`, `responsibilityType`, `scopeType`, `scopeId`, `isActive`, `activeNow`, `page`, `limit`.	`200 { responsibilityAssignments, pagination }`	Always scoped by route `institutionId`; `activeNow=true` or `activeNow=false` is evaluated by assignment status and date window.
GET	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	Path `assignmentId`.	`200 { responsibilityAssignment }`	Tenant-scoped by `institutionId + assignmentId`.
PATCH	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	JSON patch `scopeType`, `scopeId`, `startAt`, `endAt`, `isActive`.	`200 { responsibilityAssignment }`	`userId` and `responsibilityType` are not changed in V1; create a new assignment instead.
DELETE	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	No body.	`200 { responsibilityAssignment }`	Soft-deactivates by setting `isActive=false` and closing `endAt` when needed.

Create/update payload safety:

- Clients cannot set `permissions`, `institutionId`, `membershipId`, `createdByUserId`, `updatedByUserId`, `role`, `userId` on update, `responsibilityType` on update, or unknown authorization fields.
- `Membership.role` is never changed by these APIs. Head, Coordinator, and Mentor are assignment types only.
- Stored `ResponsibilityAssignment.permissions` is omitted from responses and ignored by authorization helpers.
- Duplicate active assignments are rejected for the same `institutionId + userId + responsibilityType + scopeType + scopeId`.

Frontend flow:

1. Load active institution context from `GET /institutions/me`.

2. For an admin/owner screen, list eligible instructor members with `GET /institutions/:institutionId/members?role=instructor&status=active`.
3. Let the admin pick responsibility type first, then only show allowed scopes for that type.
4. Load scope options from existing tenant routes: departments for `department`, class levels for `classLevel`, classes for `class`, sections for `section`, or use the current institution id for `institution`.
5. Submit only the allowed create fields. Do not send `permissions` or try to convert the member role.
6. After create/update/delete, refresh `GET /institutions/:institutionId/responsibility-assignments` with the same filters.

Backend flow:

1. `authJwt` validates the user token, then `requireEmailVerified` and `institutionAccess()` validate active tenant access.
2. Service verifies the actor membership is active and the actor is institution owner or base `Membership.role=admin`.
3. Service validates route `institutionId`, target active instructor membership, responsibility type, allowed scope combination, same-tenant scope record, date window, and duplicate-active conflicts.
4. Service writes only safe fields and stores an empty/non-authoritative `permissions` value.
5. Response returns safe assignment fields only: `id`, `institutionId`, `userId`, `responsibilityType`, `scopeType`, `scopeId`, `isActive`, `startAt`, `endAt`, `createdAt`, `updatedAt`.

Academic Oversight (Head Read-Only)

Read-only academic visibility for admin/owner and scoped Head assignments. These routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`; the service performs owner/admin or scoped Head authorization. Coordinator and Mentor assignments do not grant these academic oversight APIs.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/academic-oversight/risk-summary`	Optional narrowing filters `departmentId`, `classLevelId`, `sectionId`, `studentMemberId`, `search`, `riskLevel`, `standing`, `page`, `limit`.	`200 { academicRiskSummary }`	Summary is derived from attendance, published assessment results, and standing/profile services where available.
GET	`/institutions/:institutionId/academic-oversight/students`	Same safe narrowing filters.	`200 { academicRiskSummary, students, pagination }`	Returns compact student rows only inside the actor's derived scope.
GET	`/institutions/:institutionId/academic-oversight/students/:studentMemberId`	Path `studentMemberId`, optional academic filters.	`200 { student }`	Detail is still read-only and same derived scope enforced.

Head scope derivation:

- `institution` scope can see students/enrollments in the same institution.
- `department` scope is derived through `Subject.departmentId -> Section.subjectId -> Enrollment.studentMemberId`.

- `classLevel` scope is derived through `Subject.classLevelId` and `SchoolClass.classLevelId -> Section -> Enrollment`.
- Query filters only narrow the derived DB scope; they cannot expand it.

Academic risk derivation:

- Attendance risk uses closed section sessions, attendance records, and `Institution.policies` thresholds such as warning/drop absence counts, late-to-absence ratio, and whether excused counts as an absence.
- Grade risk uses published assessment results and missing published result rows.
- Academic profile and standing are reused from existing academic services where available. Missing optional data returns safe empty/null fields instead of blocking the list.

Frontend flow:

1. Use these routes only for admin/owner users or users with an active Head responsibility assignment.
2. Do not send a scope claim such as "I am department head of X" as authority. The backend reads the active `ResponsibilityAssignment` records and derives the allowed students.
3. Use filters for UI narrowing only, such as department, class level, section, risk level, standing, or search.
4. Treat the response as read-only oversight. There are no actions here to drop students, override outcomes, excuse attendance, edit grades, or manage memberships.
5. For drill-down, call the student detail route with the returned `student.studentMembershipId`.

Backend flow:

1. `authJwt`, `requireEmailVerified`, and `institutionAccess()` validate authenticated tenant access.
2. Service requires an active actor membership. Owner/admin gets institution-wide read scope.
3. Non-admin actors must have an active Head assignment with the required academic permission from the backend catalog.
4. Service derives student membership ids from tenant DB records and applies any query filters as narrowing constraints.
5. Service computes read-only risk/profile/standing rows and returns compact safe data only.

Academic Years

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/academic-years`	JSON `name`, `startDate`, `endDate`, optional `status`.	`201 { academicYear }`	`institution.manage`; one active academic year per institution; normalized-name unique.
GET	`/institutions/:institutionId/academic-years`	Query `status`, `search` / `name`, `page`, `limit`.	`200 { academicYears, pagination }`	`institution.manage`.
GET	`/institutions/:institutionId/academic-years/active`	No body.	`200 { academicYear }`	`404 Active academic year not found` if none.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/academic-years/:academicYearId`	Path `academicYearId`.	`200 { academicYear }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/academic-years/:academicYearId`	JSON patch `name`, dates, `status`.	`200 { academicYear }`	Archived years cannot be updated/activated.
POST	`/institutions/:institutionId/academic-years/:academicYearId/archive`	No body.	`200 { academicYear }`	Status-only archive.

Departments

University-facing module. Department routes are tenant-scoped by `institutionId`. Department list/get currently require active institution access but no route-level `department.manage`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/departments`	JSON `name`, `code`, `description`.	`201 { department }`	`department.manage`; university-only; unique code/name.
GET	`/institutions/:institutionId/departments`	Query `status`.	`200 { total, departments }`	Institution access.
GET	`/institutions/:institutionId/departments/:departmentId`	Path `departmentId`.	`200 { department }`	Institution access.
PATCH	`/institutions/:institutionId/departments/:departmentId`	JSON patch `name`, `code`, `description`, `status`.	`200 { department }`	`department.manage`; rejects empty/no-valid-field patch.
DELETE	`/institutions/:institutionId/departments/:departmentId`	No body.	`200 { message }`	`department.manage`; TODO/VERIFY destructive cascading expectations.

Classes / Class Levels

School-only structure module. No delete routes exist for class levels, school classes, or class assignments.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/class-levels`	JSON `name`, `code`, `description`.	`201 { classLevel }`	`class.manage`; school-only; unique code/name.
GET	`/institutions/:institutionId/class-levels`	Query `status`.	`200 { total, classLevels }`	`class.manage`.
GET	`/institutions/:institutionId/class-levels/:classLevelId`	Path `classLevelId`.	`200 { classLevel }`	`class.manage`.
PATCH	`/institutions/:institutionId/class-levels/:classLevelId`	JSON patch `name`, `code`, `description`, `status`.	`200 { classLevel }`	`class.manage`.
POST	`/institutions/:institutionId/classes`	JSON `classLevelId`, `name`, `code`.	`201 { schoolClass }`	Active class level required.
GET	`/institutions/:institutionId/classes`	Query `status`, `classLevelId`.	`200 { total, schoolClasses }`	`status` supports `active`, `archived`, or `all`.
GET	`/institutions/:institutionId/classes/:classId`	Path `classId`.	`200 { schoolClass }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/classes/:classId`	JSON patch `name`, `code`, `status`.	`200 { schoolClass }`	`classLevelId` is not changed after create.
POST	`/institutions/:institutionId/classes/:classId/students`	JSON `semesterId`, `studentMemberId`.	`201 { assignment }`	Creates/reactivates assignment and syncs matching school section enrollments.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/classes/:classId/students`	Query `semesterId`, `status`.	`200 { assignments }`	Default status is `assigned`.
GET	`/institutions/:institutionId/classes/:classId/students/:assignmentId`	Path `assignmentId`.	`200 { assignment }`	Tenant/class scoped.
PATCH	`/institutions/:institutionId/classes/:classId/students/:assignmentId/status`	JSON `status` (`assigned` or `removed`).	`200 { assignment }`	Removal marks matching school enrollments dropped.

Subjects

`Subject` is shared. University subjects are scoped through `departmentId`; school subjects are scoped through `classLevelId`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/departments/:departmentId/subjects`	JSON `name`, `code`, `description`, `creditHours`, `prerequisiteSubjectIds`, `status`.	`201 { subject }`	University subject; `creditHours` required; prerequisites tenant-scoped and deduped.
POST	`/institutions/:institutionId/class-levels/:classLevelId/subjects`	JSON `name`, `code`, `description`, `status`.	`201 { subject }`	School subject; rejects `creditHours` and department scope.
GET	`/institutions/:institutionId/subjects`	Query `departmentId`, `classLevelId`, `active`.	`200 { total, subjects }`	`subject.manage`.
GET	`/institutions/:institutionId/subjects/:subjectId`	Path `subjectId`.	`200 { subject }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/subjects/:subjectId`	JSON patch subject fields.	`200 { subject }`	Institution type controls allowed fields.
DELETE	`/institutions/:institutionId/subjects/:subjectId`	No body.	`200 { message }`	TODO/VERIFY linked section/assessment delete guards.

Memberships / Users

`User` is auth identity only. `Membership` is institution access + role. `MemberProfile` is personal/member info.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/members`	JSON or multipart: `email`, `role`, `profile`; optional `profilePicture`.	`201 { member }`	`membership.manage`; supports text/plain JSON and multipart `payload` / `profile` JSON strings.
GET	`/institutions/:institutionId/members`	Query `role`, `status`, `search`, `page`, `limit`.	`200 { total, members }`	List response is summary shape.
GET	`/institutions/:institutionId/members/:membershipId`	Path `membershipId`.	`200 { member }`	Detail response includes profile details.
PATCH	`/institutions/:institutionId/members/:membershipId`	JSON or multipart profile patch; optional `profilePicture`.	`200 { member }`	Uploaded file cleaned up when service rejects.
PATCH	`/institutions/:institutionId/members/:membershipId/status`	JSON `status` (`active` or `suspended`).	`200 { member }`	Does not change role.

Sections

Section list/detail uses route-level `section.read` plus service-level role-aware scoping.

Method	Path	Request / query	Success	Auth / scoping notes
POST	`/institutions/:institutionId/sections`	JSON `semesterId`, `subjectId`, `schoolClassId`, `code`, `title`, `instructorMemberId`, `capacity`, `schedule`, `status`.	`201 { section }`	`section.manage`; validates semester teaching days and room conflicts; school sections require `schoolClassId` and backfill class roster enrollments.
GET	`/institutions/:institutionId/sections`	Query `semesterId`, `subjectId`, `instructorMemberId`, `departmentId`, `schoolClassId`, `status`.	`200 { sections }`	Owner/admin broad; instructors only own sections; coordinator via `section.read_scope`; students blocked.
GET	`/institutions/:institutionId/sections/:sectionId`	Path `sectionId`.	`200 { section }`	Same read scoping as list; scoped readers get safer detail without creator audit metadata.
PATCH	`/institutions/:institutionId/sections/:sectionId`	JSON patch section fields.	`200 { section }`	`section.manage`; room conflict and schedule validation apply.
DELETE	`/institutions/:institutionId/sections/:sectionId`	No body.	`200 { message }`	TODO/VERIFY delete guards for linked enrollment/attendance data.

Section Sessions

Routes use `institutionAccess()` and service-level authorization. Allowed access includes admin/owner, parent section instructor, and selected derived coordinator responsibility scopes.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/sessions`	JSON `sessionType`, `sessionDate`, `startTime`, `endTime`, `room`.	`201 { sectionSession }`	`sessionType` supports `lecture` or `lab`; date must be inside semester and teaching days.
GET	`/institutions/:institutionId/sections/:sectionId/sessions`	Query `fromDate`, `toDate`, `status`, `attendanceState`, `page`, `limit`.	`200 { items, pagination }`	Uses coordinator `sectionSession.read_scope` for scoped responsibility access.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/generate`	Optional JSON `fromDate`, `toDate`.	`200 { message, generatedCount, skippedCount, sectionSessions }`	Generates from recurring section schedule; skips off/exam dates and existing sessions.
GET	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId`	Path `sectionSessionId`.	`200 { sectionSession }`	Includes lightweight `section` and `subject`.
PATCH	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/status`	JSON `status`, optional `note`.	`200 { sectionSession }`	Dedicated status endpoint; cancellation emails are best-effort to active enrolled students.
PATCH	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId`	JSON patch non-status fields.	`200 { sectionSession }`	Rejects status metadata updates.

Attendance

Attendance is anchored to `SectionSession`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/open`	No body.	`200 { sectionSession }`	Opens attendance; cancelled/closed sessions rejected.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/qr-token`	No body.	`200 { qrToken, sectionSession }`	Token expires after 20 seconds; token call replaces prior token.
PUT	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/manual`	JSON `entries[]` with `enrollmentId`, `attendanceStatus`, optional `note`.	`200 { attendanceRecords }`	Status must be `present`, `absent`, `late`, or `excused`; duplicate enrollment ids rejected.
GET	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/roster`	Query `filter`.	`200 { attendanceRoster }`	Filters: `all`, `attended`, `not_attended`, `present`, `absent`, `late`, `excused`, `pending`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/close`	No body.	`200 { sectionSession }`	Marks missing active enrollments absent.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/check-in`	JSON `token`.	`200 { attendanceRecord }`	Student-only; active enrollment required; same-day QR only; present/late computed by grace window.
GET	`/institutions/:institutionId/sections/:sectionId/attendance/analytics`	Query `fromDate`, `toDate`.	`200 { attendanceAnalytics }`	Closed non-cancelled sessions only; attendance risk is response-only and does not change enrollments/outcomes.
GET	`/institutions/:institutionId/students/me/attendance-history`	Query `sectionId`, `fromDate`, `toDate`, `status`.	`200 { attendanceHistory, summary }`	Student self route.

Enrollments / Self-Enrollment

Shared `Enrollment` model; school enrollments are backend-managed through class roster sync.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/enrollments`	JSON `studentMemberId`.	`201 { enrollment }`	`section.manage`; blocked for school sections.
GET	`/institutions/:institutionId/sections/:sectionId/enrollments`	No query.	`200 { enrollments }`	Section-scoped.
GET	`/institutions/:institutionId/sections/:sectionId/enrollments/:enrollmentId`	Path `enrollmentId`.	`200 { enrollment }`	Tenant + section scoped.
PATCH	`/institutions/:institutionId/sections/:sectionId/enrollments/:enrollmentId/status`	JSON `status` (`enrolled` or `dropped`).	`200 { enrollment }`	Use status update to re-enroll.
GET	`/institutions/:institutionId/students/me/self-enrollment/sections`	Query `availableOnly`.	`200 { sections }`	Student-only, university-only active-semester browse.
GET	`/institutions/:institutionId/students/me/self-enrollment/sections/:sectionId/eligibility`	No body.	`200 { eligibility }`	Preview route with blocking reason codes.
POST	`/institutions/:institutionId/students/me/self-enrollment/submissions`	JSON `sectionIds[]`.	`201 { enrollments, submission }`	Validates whole selected set before writes; duplicate same-subject and min/max load rules.

Assessments / Results

Assessment routes use service-level section-derived authorization. Student self routes expose only current student data.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/assessments`	JSON `title`, `description`, `type`, `assessmentDate` or `sectionSessionId`, `maxScore`, `weight`, `status`.	`201 { assessment }`	Snapshots active enrollments into result rows.
GET	`/institutions/:institutionId/sections/:sectionId/assessments`	Query `status`.	`200 { assessments }`	Staff/scoped read.
GET	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId`	Path `assessmentId`.	`200 { assessment }`	Tenant + section scoped.
PATCH	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId`	JSON patch allowed fields.	`200 { assessment }`	Archived read-only; only draft assessments can be updated.
PATCH	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/status`	JSON `status`.	`200 { assessment }`	Status supports `draft`, `announced`, `published`, or `archived`; emails best-effort on announced/published.

Method	Path	Request / query	Success	Notes
PUT	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/results`	JSON `entries[]` with `enrollmentId`, `score?`, `note?`.	`200 { assessmentResults }`	Score numeric, `0..maxScore`; unique enrollment ids.
GET	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/results`	No body.	`200 { assessmentResults }`	Staff/scoped read.
GET	`/institutions/:institutionId/sections/:sectionId/results`	No body.	`200 { sectionResults }`	Published-only weighted section results.
GET	`/institutions/:institutionId/students/me/assessments`	No body.	`200 { assessments }`	Student sees announced/published metadata; no draft/archived.
GET	`/institutions/:institutionId/students/me/results`	Query `sectionId`, `semesterId`.	`200 { results }`	Student sees published results only.

Homework / Projects

Homework is separate from AssessmentResult. Optional `assessmentId` links graded work but homework submissions do not become scores.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/homeworks`	JSON `title`, `description`, `type`, `dueAt`, `sectionSessionId`, `assessmentId`, `submissionMode`, `allowLate`, `status`.	`201 { homework }`	Assignment emails best-effort when created as assigned.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks`	Query `status`, `type`, `submissionMode`, `assessmentId`, `page`, `limit`.	`200 { homeworks, pagination }`	Staff/scoped section access.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId`	Path `homeworkId`.	`200 { homework }`	Staff/scoped section access.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId`	JSON patch homework fields.	`200 { homework }`	`submissionMode` cannot change after submissions exist.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/status`	JSON `status`.	`200 { homework }`	Emails best-effort on `draft -> assigned`.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions`	No body.	`200 { homework, submissions }`	Roster combines active enrollments with existing submissions.
PUT	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/enrollments/:enrollmentId/submission`	JSON `status`, `instructorFeedback`.	`200 { submission }`	Offline/manual mode only.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions/:submissionId`	JSON `status`, `instructorFeedback`.	`200 { submission }`	Staff submission update.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions/:submissionId/file`	No body.	File download	Private file authorization before `res.download`.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submission`	No body.	`200 { homework, submission }`	Student self route.
POST	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submission`	multipart field `submissionFile`, optional `studentNote`.	`201 { submission }`	Student-only; requires assigned `online_upload`; late blocked unless allowed.
GET	`/institutions/:institutionId/students/me/homeworks`	Query `status`, `type`, `dueFrom`, `dueTo`, `sectionId`, `page`, `limit`.	`200 { homeworks, pagination }`	Student sees assigned/closed items for active enrollments.

Section Materials

PDF-only private section resources, separate from Homework.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/materials`	multipart `materialFile`, plus `title`, `description`, `materialType`, `sectionSessionId`, `status`.	`201 { material }`	PDF required; private storage.
GET	`/institutions/:institutionId/sections/:sectionId/materials`	Query `status`, `materialType`, `page`, `limit`.	`200 { materials, pagination }`	Staff/scoped section access.
GET	`/institutions/:institutionId/sections/:sectionId/materials/:materialId`	Path `materialId`.	`200 { material }`	Staff/scoped detail.
PATCH	`/institutions/:institutionId/sections/:sectionId/materials/:materialId`	multipart optional `materialFile` plus patch fields.	`200 { material }`	Replacing file removes old private file only after DB update.
PATCH	`/institutions/:institutionId/sections/:sectionId/materials/:materialId/status`	JSON `status`.	`200 { material }`	Status supports `draft`, `published`, or `archived`; archived restore deferred.
GET	`/institutions/:institutionId/sections/:sectionId/materials/:materialId/file`	No body.	File download	Student can download only published material for active enrolled section.
GET	`/institutions/:institutionId/students/me/materials`	Query `sectionId`, `materialType`, `page`, `limit`.	`200 { materials, pagination }`	Student self list, published only.

Academic Profiles / Policies / Standing

Academic profile facade delegates by `Institution.type`.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/students/me/academic-profile`	Query `academicYearId`, `semesterId`.	`200 { academicProfile }`	Student self only; school currently requires `academicYearId`.
GET	`/institutions/:institutionId/students/:studentMembershipId/academic-profile`	Query `academicYearId`, `semesterId`.	`200 { academicProfile }`	Route uses `membership.manage`; providers also enforce school `class.manage` / university `section.manage` style access.
POST	`/institutions/:institutionId/academic-policies`	JSON university policy config.	`201 { academicPolicy }`	University-only; semester-effective.
GET	`/institutions/:institutionId/academic-policies`	Query `status`.	`200 { academicPolicies }`	`status` supports `draft`, `active`, `archived`, or `all`.
GET	`/institutions/:institutionId/academic-policies/:policyId`	Path `policyId`.	`200 { academicPolicy }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/academic-policies/:policyId`	JSON patch university policy config.	`200 { academicPolicy }`	Validates repeatPolicy and academicStandingRules.
GET	`/institutions/:institutionId/academic-standing/university/summary`	Query `standing`, `minGpa`, `maxGpa`, `semesterId`, `departmentId`, `search`, `page`, `limit`.	`200 { standingSummary }`	Admin/owner institution-wide; guarded by `section.manage`; university-only.
GET	`/institutions/:institutionId/academic-standing/university/students`	Same filters.	`200 { standingSummary, students, pagination }`	Department filtering derived through Subject.
GET	`/institutions/:institutionId/instructors/me/academic-standing/summary`	Same filters.	`200 { standingSummary }`	Instructor-scoped to assigned sections and active enrolled students.
GET	`/institutions/:institutionId/instructors/me/academic-standing/students`	Same filters.	`200 { standingSummary, students, pagination }`	Standing is live/read-only, not stored.

School Academic Policies / Report Cards / Final Outcomes

School policy and outcomes are percentage/promotion based and stay separate from university GPA/credit-hour behavior.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/academic-policies/school`	JSON school policy config.	`201 { schoolAcademicPolicy }`	School-only; requires real non-archived `appliesTo.academicYearId`.
GET	`/institutions/:institutionId/academic-policies/school`	Query `status`, `academicYearId`, `schoolStageId`, `classLevelId`.	`200 { schoolAcademicPolicies }`	Resolver supports academic-year/stage/class-level fallback.
GET	`/institutions/:institutionId/academic-policies/school/:policyId`	Path `policyId`.	`200 { schoolAcademicPolicy }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/academic-policies/school/:policyId`	JSON patch school policy config.	`200 { schoolAcademicPolicy }`	Boolean fields must be booleans; semester weights total 100.
POST	`/institutions/:institutionId/academic-policies/school/:policyId/archive`	No body.	`200 { schoolAcademicPolicy }`	Status-only lifecycle.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/semesters/:semesterId/report-card`	No body.	`200 { reportCard }`	`class.manage`; read-only.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/report-card`	No body.	`200 { reportCard }`	Includes live promotion decision explanation.
GET	`/institutions/:institutionId/school-outcomes/classes/:schoolClassId/academic-years/:academicYearId/promotion-summary`	No body.	`200 { promotionSummary }`	Current MVP unpaginated.
GET	`/institutions/:institutionId/school-outcomes/academic-years/:academicYearId/promotion-dashboard/summary`	Query admin dashboard filters.	`200 { promotionSummary }`	Uses final outcomes when present, live calculation otherwise.
GET	`/institutions/:institutionId/school-outcomes/academic-years/:academicYearId/promotion-dashboard/students`	Query `decision`, `schoolClassId`, `classLevelId`, `complete`, `hasMissingResults`, `finalized`, `search`, `page`, `limit`.	`200 { promotionSummary, students, pagination }`	Admin/staff full promotion dashboard.
GET	`/institutions/:institutionId/instructors/me/school-outcomes/academic-years/:academicYearId/promotion-dashboard/summary`	Query instructor-safe filters.	`200 { sectionOutcomeSummary }`	Instructor assigned-section only; rejects `decision` and `finalized`.
GET	`/institutions/:institutionId/instructors/me/school-outcomes/academic-years/:academicYearId/promotion-dashboard/students`	Query `schoolClassId`, `classLevelId`, `complete`, `hasMissingResults`, `search`, `page`, `limit`.	`200 { sectionOutcomeSummary, sectionOutcomes, pagination }`	No full yearly decision exposed.
POST	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/finalize`	Optional JSON `override.finalDecision`, `override.reason`.	`201 { schoolFinalOutcome }`	Does not move student classes.
PATCH	`/institutions/:institutionId/school-outcomes/final-outcomes/:outcomeId/override`	JSON `finalDecision`, `reason` or `override` object.	`200 { schoolFinalOutcome }`	Cancelled outcomes cannot be overridden.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/final-outcome`	No body.	`200 { schoolFinalOutcome }`	`404` if no final outcome exists.

Semesters

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/semesters`	JSON `academicYearId?`, `name`, `startDate`, `endDate`, `teachingDays`, `weeksCount`, `calendarOverrides`, `examDates`, `status`.	`201 { semester }`	`semester.manage`; one active semester per institution.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/semesters`	Query `status`.	`200 { total, semesters }`	`semester.manage`.
GET	`/institutions/:institutionId/semesters/active`	No body.	`200 { semester }`	Institution access only.
GET	`/institutions/:institutionId/semesters/:semesterId`	Path `semesterId`.	`200 { semester }`	Detail includes weeks.
PATCH	`/institutions/:institutionId/semesters/:semesterId`	JSON patch allowed semester fields.	`200 { semester }`	Optional `academicYearId` must reference same-tenant non-archived year.
DELETE	`/institutions/:institutionId/semesters/:semesterId`	No body.	`200 { message }`	TODO/VERIFY linked data delete guards.

Examples By Module

Auth

```
POST /auth/login
Content-Type: application/json

{ "email": "admin@example.test", "password": "Password123" }
```

```
{
  "accessToken": "<JWT_TOKEN>",
  "user": {
    "id": "507f1f77bcf86cd799439011",
    "email": "admin@example.test",
    "status": "active",
    "isEmailVerified": true
  }
}
```

Institution

```
GET /institutions/me
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "user": { "id": "507f1f77bcf86cd799439011", "email": "admin@example.test" },
  "hasInstitutions": true,
  "activeInstitution": {
    "institutionId": "507f1f77bcf86cd799439011",
    "name": "Example School",
    "type": "school",
    "role": "admin"
  },
```

```
  "nextStep": "build_academic_structure"
}
```

Billing

```
POST /institutions/507f1f77bcf86cd799439011/billing/checkout-session
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "checkoutSession": {
    "id": "cs_test_placeholder",
    "url": "https://checkout.stripe.com/c/pay/cs_test_placeholder"
  }
}
```

Responsibility Assignments

```
POST /institutions/507f1f77bcf86cd799439011/responsibility-assignments
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "userId": "507f1f77bcf86cd799439023",
  "responsibilityType": "head",
  "scopeType": "department",
  "scopeId": "507f1f77bcf86cd799439041",
  "startAt": "2026-09-01T00:00:00.000Z",
  "isActive": true
}
```

```
{
  "responsibilityAssignment": {
    "id": "507f1f77bcf86cd799439051",
    "institutionId": "507f1f77bcf86cd799439011",
    "userId": "507f1f77bcf86cd799439023",
    "responsibilityType": "head",
    "scopeType": "department",
    "scopeId": "507f1f77bcf86cd799439041",
    "isActive": true,
    "startAt": "2026-09-01T00:00:00.000Z",
    "endAt": null
  }
}
```

```
GET /institutions/507f1f77bcf86cd799439011/responsibility-assignments?
responsibilityType=head&activeNow=true
```

```
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "responsibilityAssignments": [
    {
      "id": "507f1f77bcf86cd799439051",
      "userId": "507f1f77bcf86cd799439023",
      "responsibilityType": "head",
      "scopeType": "department",
      "scopeId": "507f1f77bcf86cd799439041",
      "isActive": true
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Academic Year

```
POST /institutions/507f1f77bcf86cd799439011/academic-years
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "name": "2026-2027", "startDate": "2026-09-01", "endDate": "2027-06-30", "status": "active" }
```

```
{ "academicYear": { "id": "507f1f77bcf86cd799439011", "name": "2026-2027", "status": "active" } }
```

Members

```
POST /institutions/507f1f77bcf86cd799439011/members
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

payload={"email":"student@example.test","role":"student","profile":
{"firstName":"Sara","lastName":"Ali","nationalNumber":"S-100"}}
profilePicture=@avatar.png
```

```
{ "member": { "id": "507f1f77bcf86cd799439011", "role": "student", "status": "active" } }
```

Sections

```
POST /institutions/507f1f77bcf86cd799439011/sections
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

```
{
  "semesterId": "507f1f77bcf86cd799439011",
  "subjectId": "507f1f77bcf86cd799439011",
  "code": "MATH-7A-1",
  "title": "Math Grade 7 A",
  "capacity": 30,
  "schedule": [
    { "day": "monday", "startTime": "09:00", "endTime": "10:30", "room": "A101", "sessionType":
"lecture" }
  ],
  "status": "active"
}
```

```
{ "section": { "id": "507f1f77bcf86cd799439011", "code": "MATH-7A-1", "status": "active" } }
```

Attendance

```
PUT
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/sessions/507f1f77bcf86cd799
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "entries": [{ "enrollmentId": "507f1f77bcf86cd799439011", "attendanceStatus": "present" }] }
```

```
{ "attendanceRecords": [{ "id": "507f1f77bcf86cd799439011", "attendanceStatus": "present",
"source": "manual" }] }
```

Assessments / Results

```
PUT
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/assessments/507f1f77bcf86cd
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "entries": [{ "enrollmentId": "507f1f77bcf86cd799439011", "score": 87, "note": "Checked" }] }
```

```
{ "assessmentResults": [{ "enrollmentId": "507f1f77bcf86cd799439011", "score": 87, "note":
"Checked" }] }
```

Homework

```
POST
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/homeworks/507f1f77bcf86cd79
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

studentNote=Completed
submissionFile=@solution.pdf
```


```
{ "submission": { "id": "507f1f77bcf86cd799439011", "status": "submitted", "originalFilename":
"solution.pdf" } }
```


Section Materials

```
POST /institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/materials
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

title=Week 1 Notes
materialType=lecture_pdf
status=published
materialFile=@week-1.pdf
```


```
{ "material": { "id": "507f1f77bcf86cd799439011", "title": "Week 1 Notes", "status": "published"
} }
```


Academic Oversight

```
GET /institutions/507f1f77bcf86cd799439011/academic-oversight/students?
riskLevel=high&page=1&limit=20
Authorization: Bearer <JWT_TOKEN>
```


```
{
  "academicRiskSummary": {
    "totalStudents": 1,
    "highRiskCount": 1,
    "warningRiskCount": 0,
    "lowRiskCount": 0
  },
  "students": [
    {
      "student": {
        "studentMembershipId": "507f1f77bcf86cd799439071",
        "name": "Sara Ali",
```

```
        "email": "sara@example.test"
      },
      "riskLevel": "high",
      "attendanceRisk": { "riskLevel": "high", "absenceCount": 6 },
      "gradeRisk": { "riskLevel": "low", "averagePercentage": 82 },
      "academicStanding": null,
      "academicProfile": null,
      "reasons": ["Attendance absence count reached a high-risk threshold."]
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Academic Profile

```
GET /institutions/507f1f77bcf86cd799439011/students/me/academic-profile?
academicYearId=507f1f77bcf86cd799439011
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "academicProfile": {
    "institutionType": "school",
    "student": { "studentMembershipId": "507f1f77bcf86cd799439011", "name": "Sara Ali" },
    "summary": { "complete": false, "hasMissingResults": true },
    "warnings": [{ "code": "MISSING_REQUIRED_RESULTS", "message": "Some required results are
missing." }]
  }
}
```

Postman / Test Artifacts Inspected

Postman collections/environments exist for academic profile, academic year, attendance, homework, school academic services, school promotion dashboard, section material, self-enrollment, Stripe billing, repeat policy, and academic standing.

Focused tests exist for billing, attendance, homework, section material, authorization/permissions, scoped responsibility management, Head academic oversight, section list permissions, academic profile, academic year, school academic policy/outcomes/report cards/final outcomes/promotion dashboards, university academic policy/profile/outcomes/standing, and self-enrollment.

TODO / VERIFY Summary

- Verify exact patchable `Institution.policies` fields before generated client/schema docs for `PATCH /institutions/:institutionId`.

- Verify destructive delete guards for departments, subjects, sections, and semesters before exposing irreversible UI actions.
- Verify pagination needs for section lists and class promotion summaries as tenant data grows.
- Verify stricter PDF magic-byte validation if upload security requirements increase.
- Verify future date-window overlap handling for responsibility assignments beyond the current duplicate-active guard.
- Verify future billing access gates, customer portal needs, grace periods, trials, and plan limits.
- Verify broader attendance reporting, duplicate-aware risk escalation workflows, and configurable grade-risk thresholds for academic oversight.